

EBU6304 Software Engineering

Module introduction

- **Teaching arrangements**
- **Assessments**
- **Resources and guidance**

EBU6304: Teaching Team

Module Organiser

Dr Ling Ma

ling.ma@qmul.ac.uk



Dr Gokop Goteng

g.l.goteng@qmul.ac.uk



Dr Riasat Islam

riasat.islam@qmul.ac.uk



Dr Salman Haleem

m.haleem@qmul.ac.uk



Dr Alan Wong

alan.wong@qmul.ac.uk



Module information

- **Module website:**

- QMPlus →

<https://qmplus.qmul.ac.uk/course/view.php?id=16593>

EBU6304 – Software Engineering

- Check it regularly, as we could put *extra resources*.

- **Email:**

- You are expected to check your email regularly!
 - Use your QMUL email or BUPT email ONLY

Emails to the lecturers from other accounts are ignored.

Communication

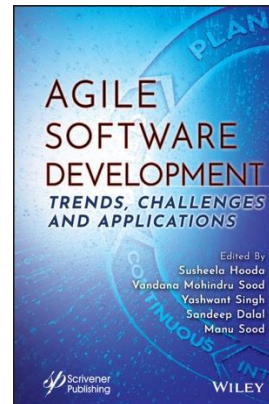
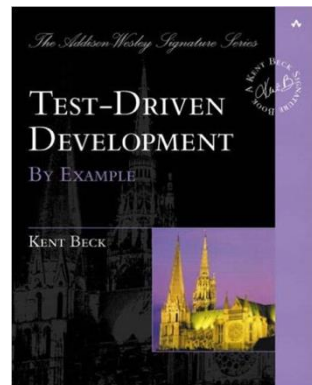
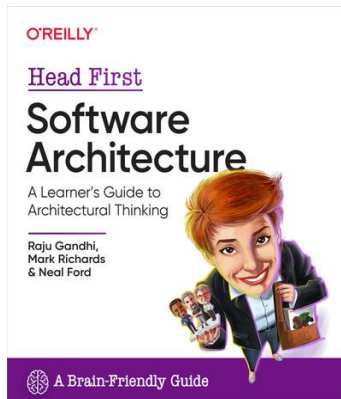
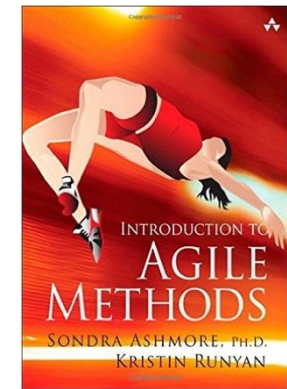
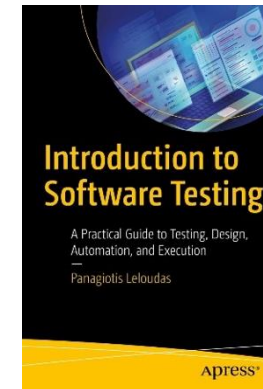
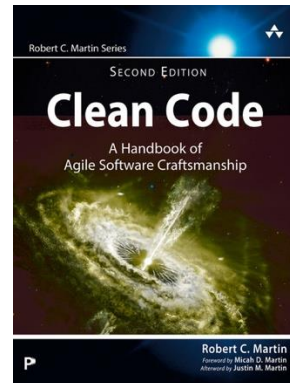
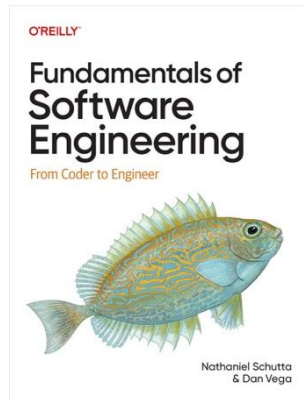
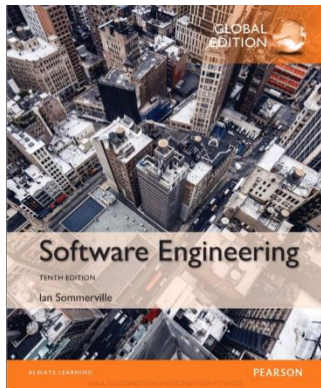
- QM+ Student Forum :
 - For all **general questions** related to the module
 - Check existing discussions before you post a new question.
 - Notification is sent to **QMUL** email ONLY

Student forum is the primary way of communication for this module

- In-person:
 - Lecture/Tutorial time
 - Office hour
- Email:
 - Use email only for personal matters, not related to others

Recommended Textbooks

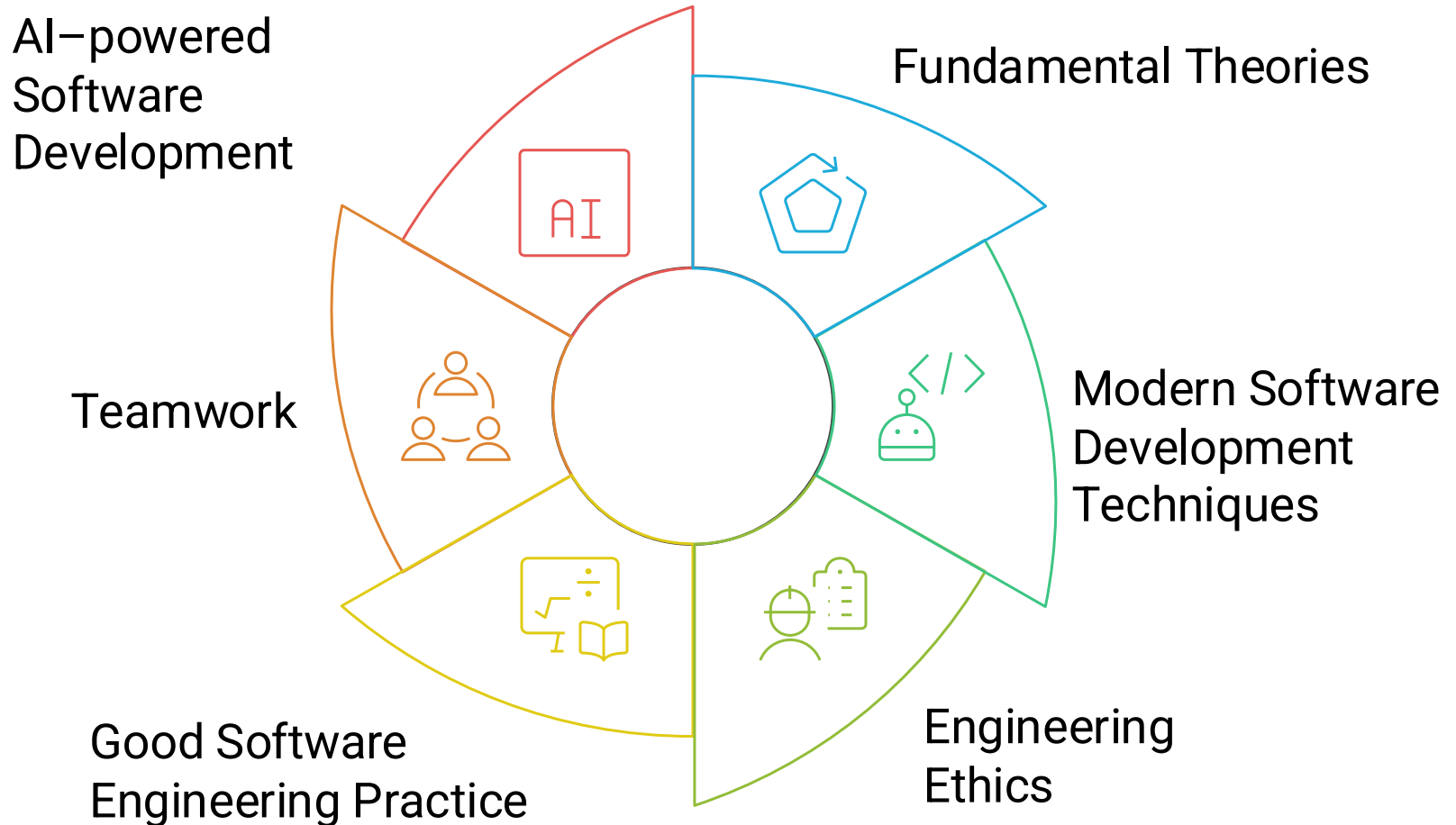
Full list can be found on Module Descriptor on QM+.
Digital versions are available at QMUL library.
<https://www.qmul.ac.uk/library/>



Online Resources

- Make use of the QMUL digital library.
<https://www.qmul.ac.uk/library/>
- There are many relevant online resources, we will recommend some and post on QM+

Module Aims and Objectives



Learning Journey

Block 1

- Software and Software Engineering
- Agile Software Development
- Requirements
- Prototyping

Block 2

- Analysis
- Design
- Implementation
- Testing

Block 3

- Software Architecture
- Project Management
- Ethics
- Quality and Security

Block 4

- AI Essentials for Software Development
- AI-powered design
- AI-powered code generation
- AI-powered testing

The use of AI

Block 1
Fundamental knowledge

Understanding
Do not use AI

Block 2
Key technologies

Applying
Do not use AI

Block 3
Advanced topics

Analysing
Use AI responsively

Block 4
AI-powered software development

Creating
Use AI responsively

Critical AI use

**You can't rely AI
for everything!**

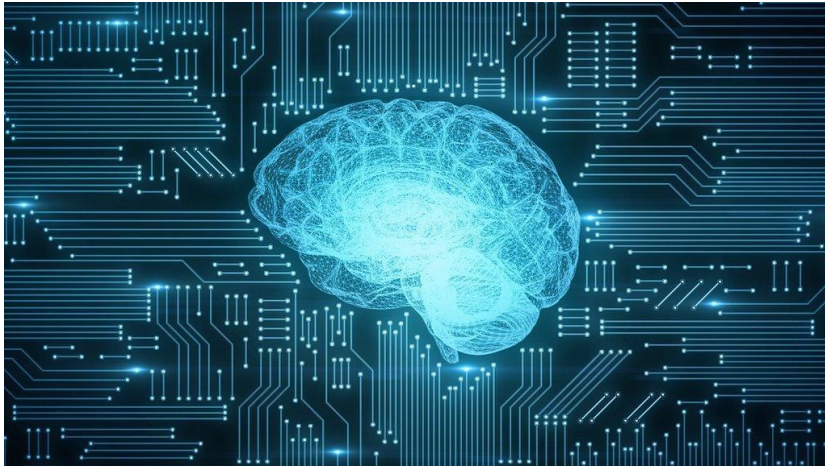


Image cc: BBC

Brain training

Logic

Structure

Reflection

Analyse

Argue

Judgement

Collaboration

Teamwork

Ethic

...

AI use guidance

- You are encouraged to use AI to **assist** with various aspects in this module:
 - Early stage: Resource finding, summary, brainstorming
 - Later stage: Story writing, prototyping, coding, and testing
- However, you should not entirely rely on AI-generated content, as AI tools are not perfect for several reasons:
 - May provide outdated or fake information
 - May lack domain-specific knowledge, introduce subtle logic errors, produce inefficient solution, or fail to fully consider ethical and privacy concerns.
 - May lack the **critical thinking** and adaptability required for real-world problem-solving

Assessments (coursework)

- **35% Group-based project**
 - Group of 6 students –form the group yourselves but all members must be in the same teaching group
 - Project time: 10 weeks
 - Assessed via staged online submission/live demo/viva, during Lab sessions
 - **Individual** marks are based on contribution, consistent progress and reflection

More details to follow
during the handout
release, at the tutorial

Assessments (exam)

- **65% Final Examination**
 - **OPEN book** exam
 - You may bring any paper material
 - Electronic devices are NOT allowed
 - Four **compulsory** questions.
 - Duration: 2 hours
 - Past papers are available on QM+ but they were **closed book** exams

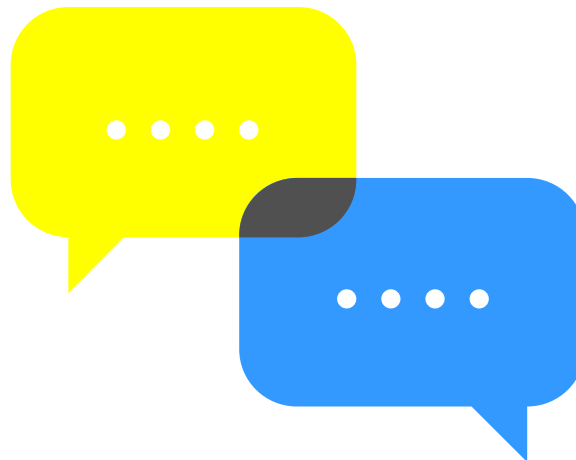
Teaching delivery

- Practice-Based Learning
 - Lectures
 - Short, focused mini-lectures (15-20 minutes each)
 - Integrated practical exercises (15-20 minutes each)
 - Directly supports preparation for the final exam
 - Tutorials
 - Hands-on support on your group CW
 - Opportunity to ask questions and get feedback
 - Labs
 - Dedicated time for group CW and personal reflection
 - Labs are week 3,4,5,6,9,10,11,12

How to Succeed?

- Lectures
 - Attend
 - Make your own notes
 - Critical thinking
 - Do the practical exercises
- Labs and Group Project
 - Teamwork
 - Engage
 - Reflection
- Open-book Exam
 - Resources
 - Understand->Apply->Create

Questions?

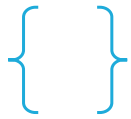


Lecture Session

Software

- **Main software types**
- **Application types from difference perspective**
- **Software quality attributes**

Software



Computer Programs

Instructions that tell the computer what to do

Code

Library



Data (non-executable)

Information that the software uses and processes

Text

Digital media



Documentation

Written materials that explain how to use and maintain the software

System Documentation

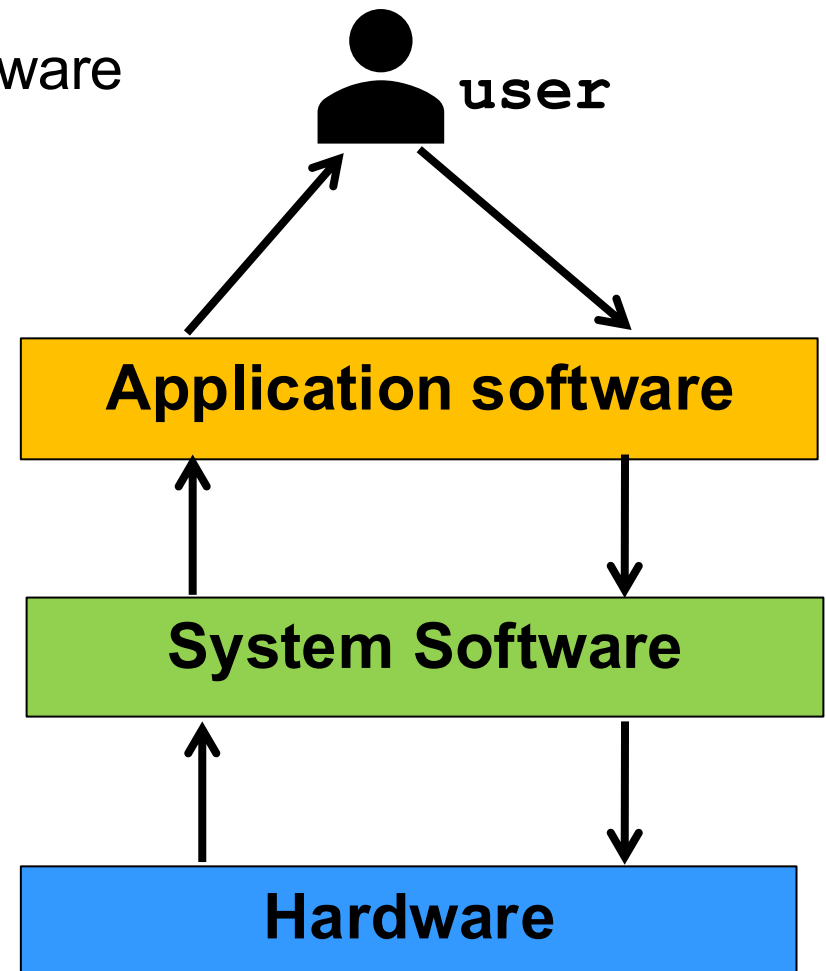
User manual

Design models

Requirements

Software types

- **System software** (manage hardware resources: for applications)
 - Operating systems
 - Device drivers
 - Utilities
- **Application software** (perform a specific task: for end users)
 - Word processing
 - Image editing
 - Social media
 - A lot more...



Application types: by device/architecture



Desktop Software

Applications for traditional personal computers/laptops



Embedded Software

Software is part of specialised devices



Mobile Applications

Apps for smartphones and tablets



Gaming Software

Entertainment applications for gaming consoles



Web Applications

Software accessed through web browsers, Client-Server



VR/AR Software

Immersive experiences for virtual and augmented reality



Server Software

Programs running on servers to manage resources



IoT Software

Software for connected devices in the Internet of Things



Cloud Software

Software as a Services (SaaS), distributed

... and more

Application types: by purpose/task



Productivity

Enhance
efficiency



Communication

Connect
with others



Browsers

Access the
internet



Entertainment

Enjoy
music, movies...



Social Media

Create, share,
and interact



Education

Teaching
and learning



Health and Fitness

Promote well-
being



Finance

Manage
money and
investments



Many others

Application types: by development

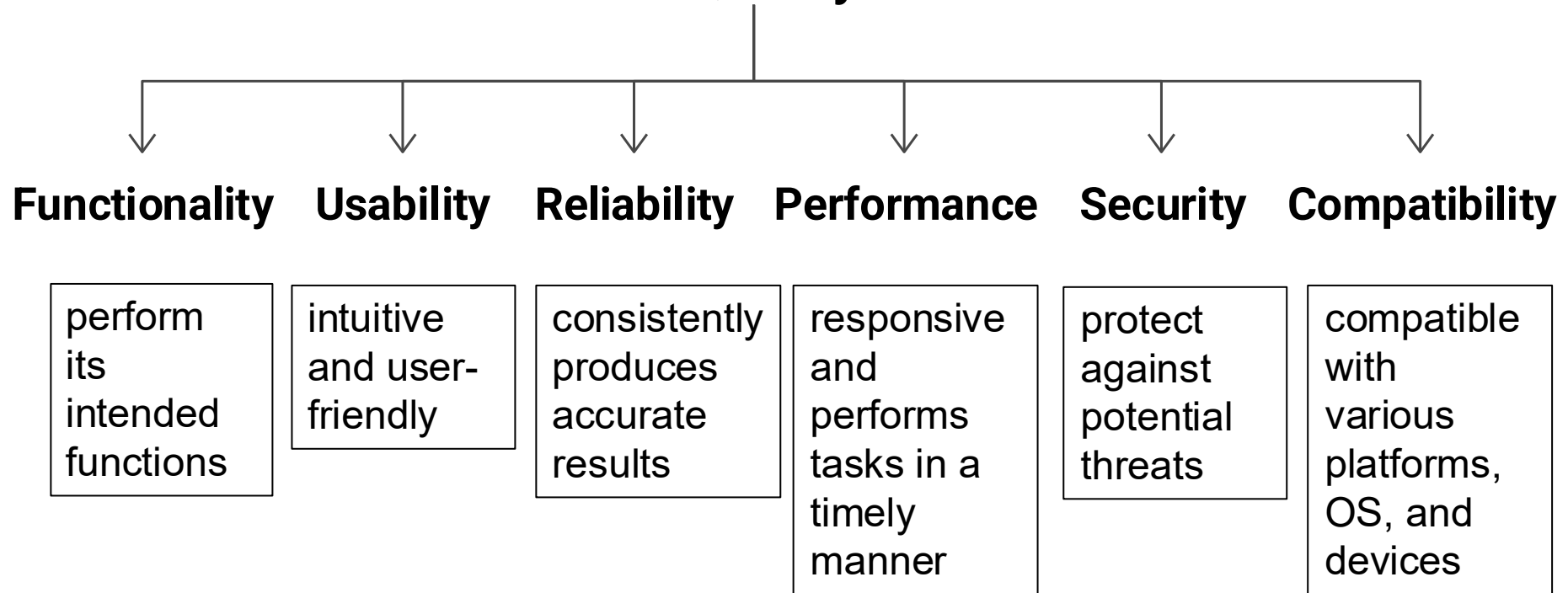
Generic Software (Off-the-Shelf)	Custom Software (Bespoke)
Developed for the general market	Developed for a specific client
Designed to meet common needs	Designed to meet specific needs
Available to many customers	Built for a single client
Owned and controlled by the vendor	Owned or licensed by the client

More and more, software companies are **starting with a generic system** and **customising it to the needs of a particular customer.**

Examples?

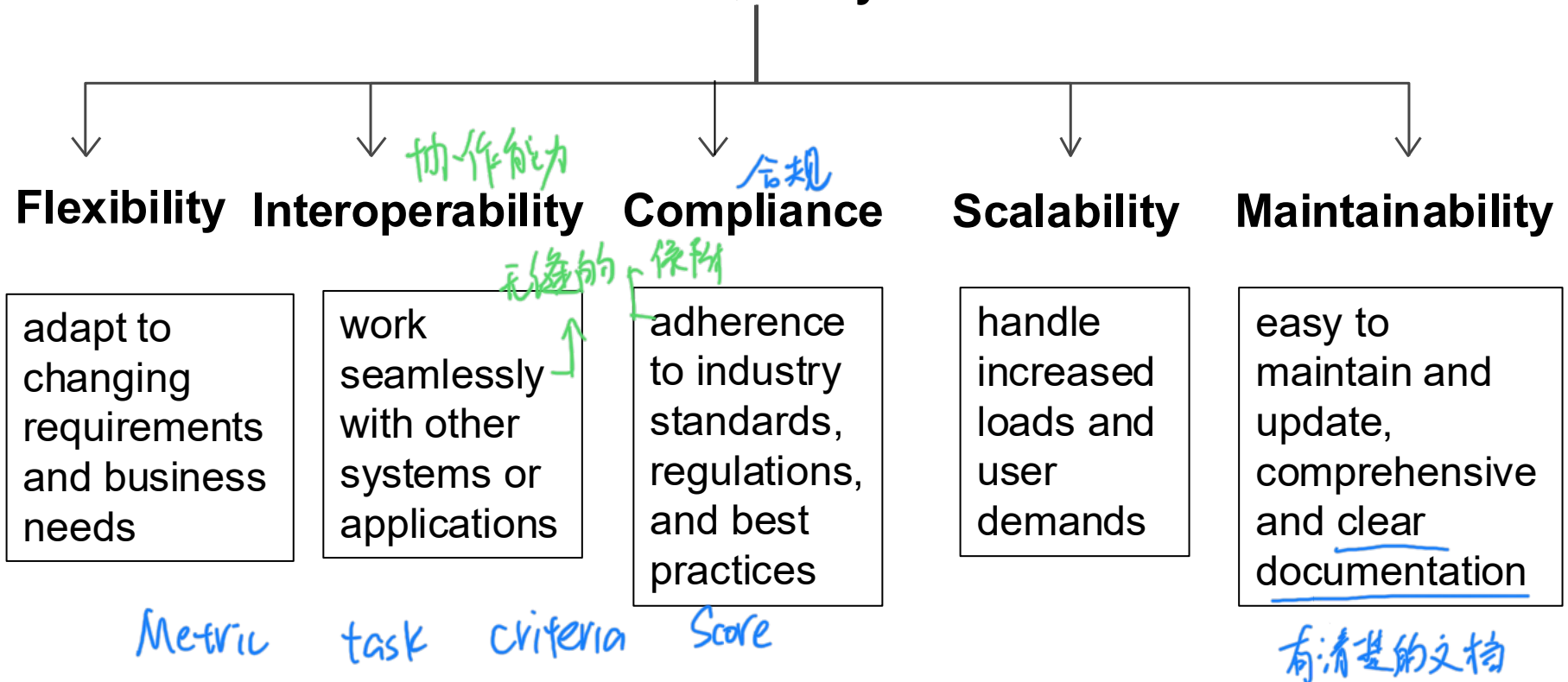
Good Software: user perspective

Software Quality Attributes

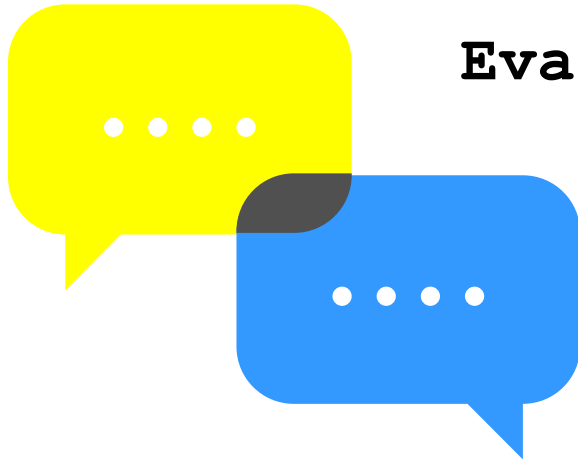


Good Software: developer perspective

Software Quality Attributes



Practical Exercises Session (15-20 minutes in class)



Evaluate a software product

Lecture Session

Software Engineering

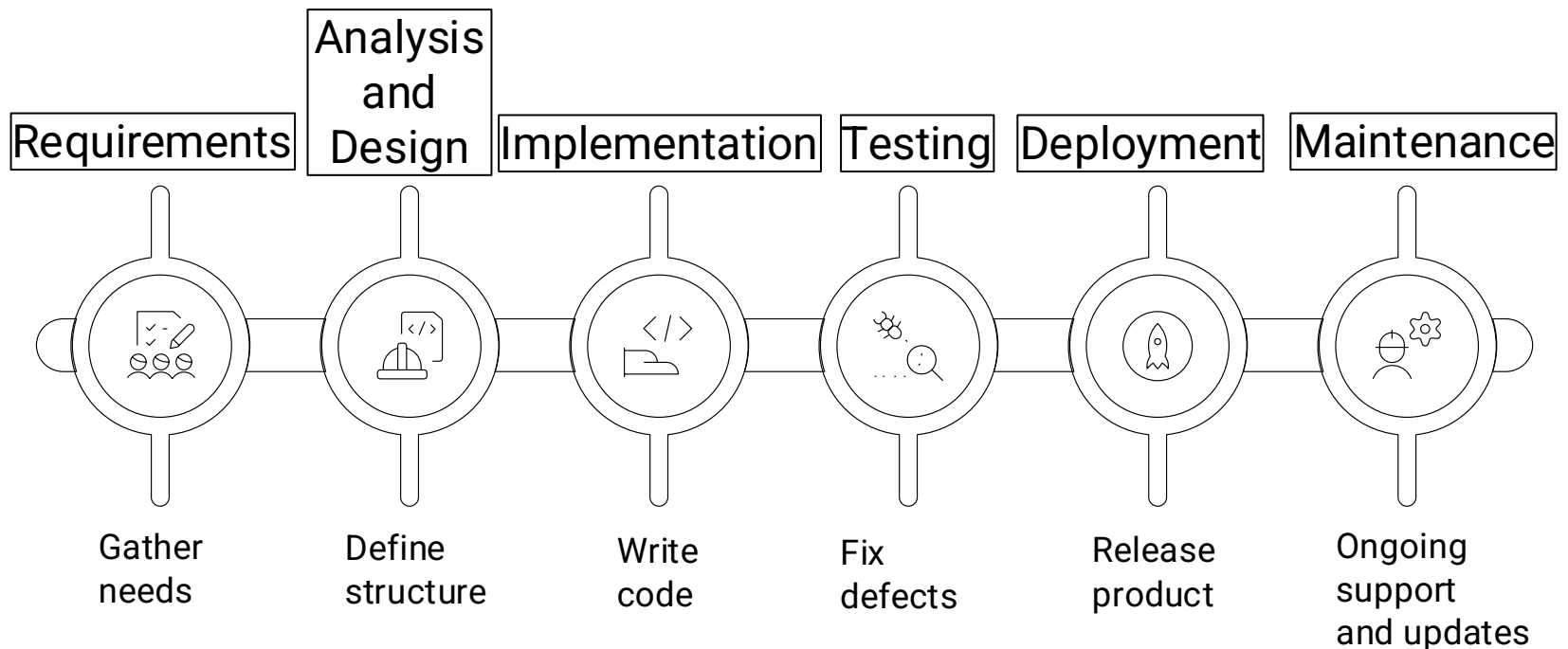
- **Key activities**
- **Importance**
- **Challenges**

Software Engineering

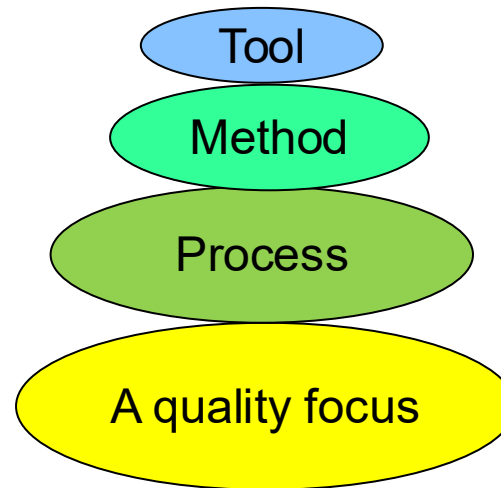
- "an engineering discipline that is concerned with all aspects of software production"—*Ian Sommerville*
- "the systematic application of scientific and technological knowledge, methods, and experience to the design, implementation, testing, and documentation of software"—*IEEE Systems and software engineering*
- "The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software"—*IEEE Standard Glossary of Software Engineering Terminology*

Software Engineering

- An engineering discipline: systematic and organised approach
- Key activities:



Software engineering layers



- Software engineering is a **layered technology**
 - Commitment to quality
 - Process: foundation layer
 - Methods: technical layer
 - Tools: support layer

Why do we need software engineering?

Time and budget

- Questions and issues:
 - Why does it take so long to get software finished?
 - Why are development costs so high?
 - Why can't we find all errors before we give the software to our customers?
 - Why do we continue to have difficulty in measuring progress as software is being developed?
 - Why do we create software that does not fulfil user requirements?

Importance of Software Engineering

- Quality Assurance
 - By following established engineering principles, build high-quality software
- Cost Efficiency
 - Reduces development cost through proper planning and design
- Scalability
 - Handle real-world complexity
- Team Collaboration
 - Supports teamwork, documentation, and maintainability

Software failures

- Large scale software development failure
 - Numerous examples of failed or seriously delayed software development projects
 - Or fail to deliver full functionality within time and budget.
 - Or disasters:
 - E.g., Health System, Airport System, Pensions System, Stock Exchange, Traffic Control System
- Small scale software development failure
 - Inconvenience
 - Financial loss
 - Waste time
 - ...



Some classic and recent case examples on QM+

Challenges in Software Engineering

- Complexity
 - distributed systems, across networks, different types of devices
- Rapid Technological Changes
 - make it difficult for software engineers to keep up with new tools, languages, and frameworks
- Changing Requirements
 - needs change during the development
- Resource
 - limited budgets and tight deadlines

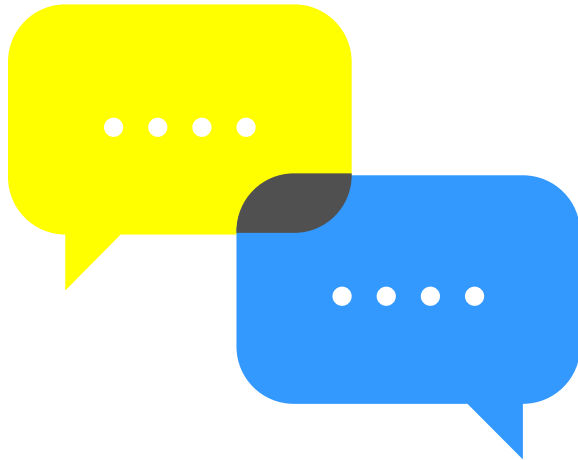
The use of AI

Critical thinking

- Benefits:
 - Help generate ideas
 - Faster prototyping
 - Increased productivity and efficiency
 - Enhanced code quality
 - Process automation
- Challenges:
 - Code reliability
 - Security risks
 - Dependence on AI
 - Intellectual property and ethical concerns

Practical Exercises Session

(15-20 minutes in class)



- Research a recent software failure case
- The change of roles of a Software Engineer in the AI era

Summary

- Software
 - Types
 - Good software attributes
- Software engineering
 - The need
 - Importance
 - Challenge